

HAPPIEST MINDS – 4.7 YEARS EXPERIENCE JAVA

INTERVIEW QUESTIONS

1. What is the purpose of `@EnableJpaRepository` annotation?

`@EnableJpaRepository` is a Spring Data annotation used to enable JPA repository scanning so that Spring Boot can:

- ✓ Detect repository interfaces
- ✓ Create proxy implementations at runtime
- ✓ Link them with the correct entity manager

✓ When do we actually use `@EnableJpaRepository`?

✓ 1. You have multiple datasources

(e.g., **MySQL** for user service and **MongoDB** for audit logs)

✓ 2. Your repositories are not under the default package

Spring Boot scans only the sub-packages of the class where `@SpringBootApplication` is placed.

✓ 3. You want custom repository scanning behavior

(custom repository base class, postfix, etc.)

2. What happens if two beans of the same type are defined without a qualifier / `@primary`?

Spring will fail to start the application and throw: **NoUniqueBeanDefinitionException**

3. How does singleton design pattern work & what types of singleton implementation exists?

✓ How Singleton Design Pattern Works

Singleton pattern ensures that:

1. Only one instance of a class is created.
2. That instance is globally accessible from anywhere in the application.

✓ How it works (simple explanation)

- The class makes its constructor private, so no other class can create objects.
- It provides a static method (usually `getInstance()`) to create or return the instance.

- The instance is stored in a private static variable.

✓ **Types of Singleton Implementations**

- ✓ 1. Eager Initialization Singleton
- ✓ 2. Lazy Initialization Singleton
- ✓ 3. Thread-Safe Singleton (Synchronized Method)
- ✓ 4. Double-Checked Locking Singleton
- ✓ 5. Bill Pugh Singleton / Inner Static Class
- ✓ 6. Enum Singleton — Best & Simplest

4.What issue can lazy loading cause in springboot RESTAPI'S?

Lazy loading can cause LazyInitializationException in Spring Boot REST APIs because associations marked as LAZY are loaded outside the Hibernate session during JSON serialization. This happens because the session is closed before the controller returns the response. It may also cause infinite recursion, performance issues (N+1 queries), and serialization errors.

5. What deployment strategies have you used in production (blue-green,canary,rolling)? Explain your approach?

✓ **Deployment Strategies I Have Used in Production**

In real-world microservices deployments, I have personally worked with:

- ✓ Blue–Green Deployment
- ✓ Canary Deployment
- ✓ Rolling Deployment

✓ **1. Blue–Green Deployment (Used for zero-downtime releases)**

Where I used it:

E-commerce microservices (User, Cart, Order) deployed on AWS Elastic Beanstalk + RDS and later Kubernetes (EKS).

✓ **Approach I followed**

- Maintain two identical environments:
 - Blue → Current production
 - Green → New version
- Deploy new application to Green environment.
- Run smoke tests and automated regression tests.
- Switch traffic from Blue → Green using:

- AWS ALB
- or Kubernetes Service selector switch

✅ Why Blue-Green?

- Zero downtime
- Instant rollback by switching traffic back to Blue
- Safe for critical services like Payments, Auth, Notification

✅ Real-time scenario

Payment service update: upgrading from Java 11 → 17.

We deployed to Green, tested payment flows, then switched ALB target groups to Green.

✅ 2. Canary Deployment (Used for reducing risk of production issues)

Where I used it:

Kubernetes-based microservices using **Istio + Argo Rollouts**, especially on high-traffic APIs like **Product Search** and **Recommendation Engine**.

✅ Approach I followed

- Deploy new version **v2** to a small percentage of traffic:
 - 5% → monitor
 - 15% → monitor
 - 50% → monitor
 - 100% → if stable
- Use **Istio traffic splitting**:

Monitored: ✅ Latency

✅ Error rates

✅ CPU/Memory

✅ Business KPIs (conversion rate)

✅ Why Canary?

- Safer than rolling updates
- Only a small percentage of users see the issue
- Metrics-driven auto rollback

- Useful for ML-based or user-sensitive changes like recommendations or pricing

✅ Real-time scenario

A new search algorithm had the potential to increase latency.

We canaried it to **10% users**, saw a spike in p95 latency → auto-rollback triggered by Argo.

✅ 3. Rolling Deployment (Used for routine updates)

Where I used it:

Daily deployments of authentication, user-profile, and notification microservices running on Kubernetes.

✅ Approach I followed

- Rolling update gradually replaces old Pods with new Pods.
- At any time:
 - maxSurge = 1
 - maxUnavailable = 0
- Kubernetes ensured: ☒ No downtime
 - ☒ No capacity reduction
 - ☒ Smooth transition

✅ Why Rolling?

- Simple
- Built-in Kubernetes strategy
- Perfect for small, safe changes:
 - UI text changes
 - Minor bug fixes
 - Log improvements
 - Patch releases

✅ Real-time example

Updating a microservice's logging library.

Rolling update applied across all pods one by one → no downtime.

Deployment Strategy	When I Use	Why
Blue-Green	Major releases, risky changes, database migrations	Instant rollback, zero downtime
Canary	Performance-sensitive or algorithm changes	Fine-grained control, production testing on small %
Rolling	Routine small releases	Simple, fast, no special infra needed

6. Are kafka communication asynchronous.Explain how it works?

Kafka communication is asynchronous, and this is one of the biggest reasons companies use Kafka in microservices.

Apache Kafka is fundamentally designed for **asynchronous, event-driven communication**.

In Kafka:

- Producers **send messages** without waiting for consumers to process them.
- Consumers **read messages later**, at their own speed.
- No real-time request/response between services.

This makes Kafka **decoupled, scalable, and fault-tolerant**.

7.If you update dependency in pom.xml but the old version is still used how to fix it?

This happens because Maven may be pulling the old version from another location such as:

- ✓ Parent POM
- ✓ Transitive dependencies
- ✓ Dependency management section
- ✓ Local Maven cache
- ✓ Spring Boot's dependency management (spring-boot-starter-parent)

To fix run : mvn dependency:tree

8. If an interface extends another interface & a lambda expression stops working, why might that happen?

A lambda expression works only with Functional Interfaces.

A Functional Interface is an interface that has:

- ✅ Exactly ONE abstract method

If an interface extends another interface and gains more than one abstract method, then:

- ❌ It is no longer a functional interface
- ❌ Lambda expressions WILL STOP working
- ❌ Compiler throws error: **Multiple non-overriding abstract methods found in interface**

9. What is lazy evaluation in Java Streams?

Lazy Evaluation means that intermediate operations in a Java Stream are NOT executed immediately.

They are only executed when a terminal operation is called.

10. What is the role of application.properties & how do you manage environment specific configurations?

application.properties is the central configuration file for a Spring Boot application. It is used to externalize configuration, meaning you *don't hardcode values* inside the application.

- ✅ What does it store?

Common configurations include:

- ✅ Server port
server.port=8081
- ✅ Database configuration
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
- ✅ Logging level
logging.level.org.springframework=DEBUG

- ✅ Why is this important?

- Central place for settings
- Change values *without changing code*
- Easier CI/CD & DevOps operations

- Avoids redeploying app for configuration updates

✓ How do you manage environment-specific configurations?

In real production setups, you will have multiple environments like:

- **dev**
- **qa / test(application-test.properties)**
- **stage / pre-prod**
- **prod**

Each environment requires **different URLs, DB configurations, credentials, message brokers, etc.**

Spring Boot provides **3 powerful ways** to manage this.

✓ 1. Using Profile-Based Application Files (Most Common)

Spring Boot supports:

- application-dev.properties
- application-qa.properties
- application-prod.properties

11. What is the difference between git rebase and git cherry-pick?

✓ 1. What is Git Rebase?

Rebase moves an entire branch on top of another branch, rewriting its commit history.

✓ 2. What is Git Cherry-Pick?

Cherry-pick copies specific commits from one branch to another.

Feature	git rebase	git cherry-pick
Purpose	Move whole branch onto another	Copy specific commits
Affects history?	✓ YES (rewrites)	✓ YES (creates new commits)
Best used for	Keeping clean linear history	Applying isolated fixes
Workflow impact	Changes many commits	Changes only chosen commits
Risk	High if used on shared branches	Low

Example	Sync feature with main	Apply one bugfix to prod
---------	------------------------	--------------------------

12. When should you use interface versus an abstract class?

Both **interface** and **abstract class** are used to achieve abstraction in Java, but the choice depends on **design needs** and **use-case requirements**.

✅ Use an Interface When...

✅ 1. You want to define capabilities/behaviors (what a class *can do*)

Interfaces define common **behaviors**, not structure.

Example:

- Runnable
- Comparable
- Serializable

These express abilities that ANY class can have.

✅ 2. You need Multiple Inheritance

A class can implement **multiple interfaces**, but extend **only one** class.

✅ 3. You want loose coupling (common in microservices & layered architecture)

✅ 4. You want to declare constants

Interfaces allow public static final constants.

✅ 5. You're designing something that represents a contract

Interfaces define "what to do" but not "how".

Example:

Java Streams, functional interfaces (Predicate, Function, etc.)

✅ Use an Abstract Class When...

✅ 1. You want to share common code among subclasses

Abstract classes allow:

- ✅ Method implementations
- ✅ Common fields
- ✅ Constructors

✅ 2. You want partial abstraction

Abstract class can contain:

- abstract methods
- non-abstract methods
- fields
- constructors

Interfaces cannot have instance fields or constructors.

✅ 3. You expect future versions to add more shared logic

Abstract classes are easier to evolve without breaking subclasses.

✅ 4. You need controlled inheritance

Only classes that extend the abstract class get the shared logic.

✅ 5. You know the class hierarchy is related

Use abstract class when:

- ✅ There is a "IS-A" relationship
- ✅ Classes share common behavior and properties

Example:

- Dog → Animal
- Truck → Vehicle
- Employee → BaseEmployee

13. ArrayList or LinkedList which is better for search & why?

ArrayList is better for search because it provides $O(1)$ random access using index-based lookup, while LinkedList requires $O(n)$ sequential traversal of nodes, making it much slower.

14. How can you implement a custom bean scope in spring?

Spring provides built-in scopes:

- singleton
- prototype
- request

- session
- application
- websocket

But you can also create your own custom scope by:

1.  Creating a class that implements `org.springframework.beans.factory.config.Scope`
2.  Registering this custom scope using a `BeanFactoryPostProcessor`
3.  Using `@Scope("yourScope")` on your beans

15. What problems occur when mutable objects are used as keys in a hashMap?

Using mutable objects as HashMap keys breaks the contract of `hashCode()` and `equals()`. If the key's fields change after insertion, the object becomes impossible to find, causing lookup failures, incorrect removals, duplicate keys, and inconsistent map behavior. Immutable keys are always recommended.

16. After a production deployment issues, what would your 1st step of investigation?

-  1. Immediately check logs & monitoring dashboards for errors (FIRST ACTION)

17. Which set implementation maintains insertion order and why would you use it?

LinkedHashSet

It is the Set implementation that preserves insertion order.

Because internally it uses:

- A HashMap for storing elements
- A doubly linked list to maintain order of insertion

The linked list tracks:

- The first element inserted
- The next
- ...and so on

So iteration order will always match the order in which elements were added.

18. How would you configure spring boot application for multiple environments like dev, test & prod?

Spring Boot provides Spring Profiles, which allow you to maintain environment-specific configurations cleanly.

You typically create:

- application-dev.properties
- application-test.properties
- application-prod.properties

Then activate the profile depending on where the application runs.

19. What is the order of precedence for spring boot configuration sources?

Highest Priority

1. Command-line arguments
2. Java System properties
3. OS Environment variables
4. External config files (/config/application.properties)
5. Internal config files (classpath:/application.properties)
6. Profile-specific files
7. Application JAR internal properties
8. Default Spring Boot properties

Lowest Priority

20. can you name some build in java exceptions?

1. Runtime Exceptions (Unchecked exceptions)

These extend RuntimeException and usually indicate programming mistakes.

Common Examples

- NullPointerException
Occurs when you access a method/field on null.
- ArrayIndexOutOfBoundsException
Accessing an array index outside its valid range.

- `ArithmeticException`
Example: divide-by-zero error.
- `ClassCastException`
Invalid type casting.
- `IllegalArgumentException`
Invalid method argument passed.
- `NumberFormatException`
Parsing invalid numeric value.
Example: `Integer.parseInt("abc")`
- `IllegalStateException`
Method invoked at an illegal or inappropriate time.
- `UnsupportedOperationException`
When an operation is not supported.
- `StringIndexOutOfBoundsException`

✅ 2. Checked Exceptions (must handle or declare)

These extend `Exception`, excluding `RuntimeException`.

Common Examples

- `IOException`
File not found, read/write failure.
- `FileNotFoundException`
Specific type of `IOException`.
- `ParseException`
Errors while parsing data (dates, formats).
- `SQLException`
Database errors via JDBC.
- `ClassNotFoundException`
When a class is not found at runtime (ClassLoader issues).
- `InterruptedException`
Thread interruption.

✅ 3. Errors (should not be handled typically)

These extend **Error** and indicate JVM-level issues.

Common Examples

- **StackOverflowError**
Infinite recursion.
- **OutOfMemoryError**
JVM heap full.
- **NoClassDefFoundError**
Class found at compile-time but missing at runtime.
- **AssertionError**

21. What agile ceremonies or processes have you followed and what benefits do they provide?

1. Sprint Planning

What happens?

- Team selects user stories from the Product Backlog.
- Breaks them into tasks.
- Estimates effort (story points).
- Defines Sprint Goal.

Benefits

-  Clear understanding of what will be delivered in the sprint
-  Aligns team on priorities
-  Reduces ambiguity and ensures realistic commitments
-  Avoids scope creep during sprint

2. Daily Stand-up (Daily Scrum)

What happens?

A short 15-minute meeting where each team member answers:

- What did I do yesterday?
- What will I do today?
- Any blockers?

Benefits

-  Improves team communication
-  Identifies blockers early

- ✓ Helps Scrum Master resolve issues quickly
- ✓ Keeps team focused and aligned daily

✓ 3. Backlog Refinement (Grooming)

What happens?

- Review upcoming backlog items
- Clarify requirements
- Add acceptance criteria
- Estimate story points
- Break down large epics

Benefits

- ✓ Ensures backlog is always clean, clear, prioritized
- ✓ Reduces confusion during Sprint Planning
- ✓ Helps Product Owner improve clarity
- ✓ Avoids “unknown tasks” during development

✓ 4. Sprint Review (Demo)

What happens?

- Team demonstrates completed features to stakeholders
- Collect feedback
- Discuss what was done vs planned

Benefits

- ✓ Transparency with stakeholders
- ✓ Early feedback → improves product quality
- ✓ Helps adjust upcoming backlog items
- ✓ Builds confidence in development progress

✓ 5. Sprint Retrospective

What happens?

Team reflects on:

- ✓ What went well
- ✓ What didn't go well

-  What improvements to implement next sprint

Benefits

-  Continuous improvement
-  Better collaboration and communication
-  Reduces repeated mistakes
-  Increases team efficiency & morale

6. Release Planning

(Used especially in microservices or larger systems)

What happens?

- Decide what features go into next release
- Coordinate with DevOps for deployment windows
- Risk assessment and dependency management

Benefits

-  Smooth deployments
-  Fewer production issues
-  Proper coordination with business

22. Have you implemented custom exceptions when would you use them

Yes, I have implemented custom exceptions in multiple Spring Boot microservices to handle **domain-specific errors**, improve **clarity**, and provide **clean, meaningful API responses**.

You create custom exceptions when:

1. You want to represent business-specific errors

Example:

- InsufficientBalanceException
- InvalidOrderStateException
- UserNotFoundException

These errors are **not covered** by Java's built-in exceptions.

2.Handle it globally using @ControllerAdvice

23. What makes interface `FunctionalInterface`

A Functional Interface in Java is an interface that has:

- ✓ Exactly ONE abstract method

This is the ONLY requirement.

This single abstract method represents the functional contract, which makes the interface eligible to be used with:

- ✓ Lambda Expressions
- ✓ Method References
- ✓ Stream operations
- ✓ Functional Programming patterns

24. What is the importance of finally block in exception handling

The finally block is used in Java to ensure that critical cleanup code always executes, whether an exception occurs or not.

It is executed:

- ✓ If an exception is thrown
- ✓ If an exception is *not* thrown
- ✓ If a return statement is executed
- ✓ Even if the code inside try or catch completes normally

Only one case where it may not run:

- ✗ When JVM shuts down abruptly (`System.exit()`, power failure, OS crash)

25. How does java manage memory allocation between stack & heap ?

Java divides memory into two main areas:

- ✓ Stack Memory → stores method calls, local variables, references
- ✓ Heap Memory → stores objects created via `new`

Both are managed by the JVM, but in different ways.

Feature	Stack	Heap
Speed	✓ Very fast	✗ Slower
Lifetime	Method-level	App-level

Allocation	Automatic	Dynamic (new)
GC	Not required	Required
Thread Safety	Yes (per thread)	No (shared)
Stores	Primitives & references	Objects